Do Test and Environmental Complexity Increase Flakiness? An Empirical Study of SAP HANA

Alexander Berndt
SAP
Germany
alexander.berndt@sap.com

Thomas Bach
SAP
Germany
thomas.bach03@sap.com

Sebastian Baltes
University of Bayreuth
Germany
sebastian.baltes@uni-bayreuth.de

Abstract

Background: Test flakiness is a major problem in the software industry. Flaky tests fail seemingly at random without changes to the code and thus impede continuous integration (CI). Some researchers argue that all tests can be considered flaky and that tests only differ in their frequency of flaky failures. This position implies that the definition of test flakiness includes failures caused by interruptions in the testing environment.

Aims: With the goal of developing mitigation strategies to reduce the negative impact of test flakiness, we study characteristics of tests and the test environment that potentially impact test flakiness.

Method: We construct two datasets based on SAP HANA's test results over a 12-week period: one based on production data of the SAP HANA CI pipeline, the other based on targeted test executions from a dedicated flakiness experiment. We conduct correlation analysis for test and test environment characteristics with respect to their influence on the frequency of flaky test failures.

Results: In our study, the average test execution time had the strongest positive correlation with the test flakiness rate ($r = 0.79$), which confirms previous studies. Potential reasons for higher flakiness include the larger test scope of long-running tests or test executions on a slower test infrastructure. We found that distributed tests had a lower flakiness rate than non-distributed tests. Interestingly, the load on the testing infrastructure was not correlated with test flakiness. The relationship between test flakiness and required resources for test execution (i.e., memory and CPU) is inconclusive.

Conclusions: Based on our findings, we conclude that splitting long-running tests can be an important measure for practitioners to cope with test flakiness. Test splitting enables parallelization of test executions and also reduces the cost of re-executions after flaky failures because the scope of the re-executed tests is narrower. Thus, splitting long-running tests into smaller tests with a narrower scope can effectively decrease the negative effects of test flakiness in complex testing environments. However, when splitting long-running tests, practitioners need to consider the potential test setup overhead of test splits.



This work is licensed under a Creative Commons Attribution International 4.0 License.

ESEM '24, October 24–25, 2024, Barcelona, Spain
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1047-6/24/10
https://doi.org/10.1145/3674805.3695407

CCS Concepts

• **Software and its engineering → Software testing and debugging**.

Keywords

Test Flakiness, Flaky Tests, Software Testing, Empirical Study, Database Management Systems, Regression Analysis

ACM Reference Format:
Alexander Berndt, Thomas Bach, and Sebastian Baltes. 2024. Do Test and Environmental Complexity Increase Flakiness? An Empirical Study of SAP HANA. In *Proceedings of the 18th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '24), October 24–25, 2024, Barcelona, Spain.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3674805.3695407

1 Lay Abstract

Test flakiness, that is, tests failing unpredictably without changes to the code, is a significant issue in software development. It disrupts the continuous integration (CI) process, making it more difficult for teams to maintain their software systems. Some researchers suggest that all tests experience flakiness, not necessarily due to the code base, but also due to problems in the testing environment. In our study, we explore factors that might cause tests to become flaky, with the aim of helping software engineers develop strategies to reduce these random failures. We analyze two datasets derived from SAP HANA test data for a period of 12 weeks. One dataset is based on test results in the production environment, while the other is based on a special experiment designed to study flakiness. Our findings showed that tests with longer execution times were more likely to be flaky. This might be because longer tests cover more functionality or are more affected by issues in the testing infrastructure. Interestingly, distributed tests were less flaky than those running on a single machine. We discovered that the load on the testing infrastructure did not seem to influence test flakiness, and the relationship between the memory and CPU needed for a test to run and the test's flakiness was unclear. Based on these results, we recommend splitting longer tests into smaller ones. This approach not only allows for parallel test execution but also makes it easier to identify and fix flaky failures. However, it is important to balance the benefits of smaller tests against potential overheads.

2 Introduction

Test flakiness is a major problem in the software industry. Flaky tests yield different results when executed multiple times on the same code version. Thus, flaky tests impede continuous integration, because, to automatically merge a proposed code change [9], all tests need to pass. Since software engineers aim to achieve shorter

and shorter release cycles, test flakiness has gained increasing attention from companies such as Google, Apple, Microsoft, Meta, and SAP [1, 8, 21, 22, 29].

A common strategy to mitigate the negative effects of test flakiness is to re-execute failing tests multiple times on the same code version. When the test passes in one of the re-executions, the initial failure is considered a *flaky failure* and the test is viewed as passing [4, 8, 16, 29]. However, this strategy is costly with respect to computational resources. For example, for SAP HANA's pre-submit testing in the main code line alone, 500 hours of computing time daily are dedicated to restarts caused by flaky failures. Google reports that they use up to one day of computing time for every week they spend testing [30].

Previous work highlighted that the probability of flaky failures increases with the complexity of both the executed tests and their execution environment [1, 8, 14]. For example, when tests are executed in distributed testing environments with heterogeneous hosts [8, 30], the number of flaky failures due to timeout flakiness might increase, as the increased variance of test execution times makes it harder for developers to determine appropriate timeout values [8]. When it comes to tests themselves, system tests covering a large scope with a low degree of isolation may exhibit a higher flakiness rate than smaller unit tests. However, system tests are particularly common in large-scale industrial software systems [1, 8].

The connection between test complexity and flakiness appears to be a consensus among practitioners [1, 21, 28]. Previous research found that characteristics related to test complexity are effective features for predicting whether a test is flaky [2]. Furthermore, a Google blog post reported a strong correlation between the binary size and memory usage of a test and its flakiness rate [28].

In this paper, we present a study of test flakiness in the context of a large industrial database management system, SAP HANA. First, we validate previous findings on correlations between test characteristics and flakiness. We perform a correlation analysis between the flakiness rate of a test and its average execution time, the required number of CPU threads, and the required main memory.

Based on internal discussions with practitioners at SAP, we further analyze whether tests that verify functionality in distributed environments exhibit higher flakiness rates than non-distributed tests. The assumption that distributed tests are more flaky appears intuitive because tests in distributed environments might be more susceptible to common flakiness categories such as *Async wait*, *Concurrency*, or *Network* [1, 31]. However, to our knowledge, there is no empirical evidence that this relationship exists.

In addition to the above test characteristics, we correlate two characteristics of SAP HANA's test environment with test flakiness. First, we validate previous findings that indicate a correlation between load on the testing system and flakiness [13, 27, 36]. Second, we examine the correlation between flakiness and the performance of the test execution host. More specifically, since SAP HANA's testing infrastructure is scaled out across more than 1000 heterogeneous physical hosts, we hypothesize that the flakiness rate increases on hosts with lower computational power.

To study the test and environmental characteristics mentioned above, we constructed two datasets, one arising from the productive CI environment of SAP HANA, and one from dedicated experiments to study test flakiness. In total, we obtained more than 1.5 million test results over a 12-week period.

The main contributions of the study presented in this paper are:

(1) Validation of previous findings on correlations between test (environment) characteristics and test flakiness using data from system tests of a large industrial database management system [2, 13, 26, 27, 36].
(2) A new set of test (environment) characteristics, derived from discussions with practitioners at SAP, and their correlation with test flakiness.
(3) An evaluation of the practical usefulness of our results for a large-scale industrial project.

In the following, we describe our study subject, the constructed datasets, our definition of test flakiness, and our research questions. Then, we present our results, discuss them, and conclude the paper with a discussion of threats to validity and concluding remarks.

3 Background

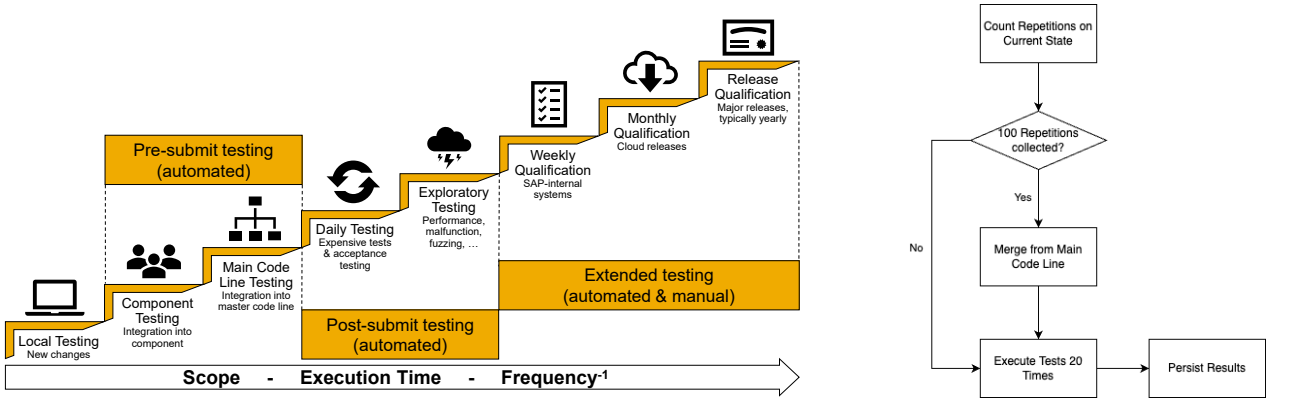
In this section, we present the study background. First, we introduce the study subject, Second, we present the datasets used. Third, we present the research questions and the methods to answer them.

3.1 Study Subject

SAP HANA, the subject of this study, is a large-scale in-memory database management system that has been developed by SAP for more than ten years [3]. To enable its use in business-critical customer scenarios, SAP HANA is extensively tested in multiple stages that vary in scope and execution frequency, as shown in Figure 1a. Bach et al. provide a more detailed overview of testing at SAP HANA in [3].

This work focuses on system tests in the so-called pre-submit testing stage, i.e., the tests that are executed before a developer's change is merged into the main code line. Since executing all pre-submit tests sequentially would take more than three days, the tests are distributed over multiple hosts and executed in parallel. For parallelization, the tests are grouped according to their required configuration of SAP HANA. In a test run, each group of tests is then executed against a dedicated installation of SAP HANA running in a Docker container. In many cases, these tests contain SQL statements to communicate with the system under test [3]. In this study, we focus on the system tests in SAP HANA's pre-submit stage. Thus, the scope of our study is approximately 800 tests that account for more than 90% of the test resource consumption in the SAP HANA pre-submit testing stage.

To cope with the immense computational demands of testing, SAP HANA's testing infrastructure is scaled out across approximately 1000 physical hosts that vary in their age, location, number of CPU threads, memory, and processor generation [3]. For every change to be merged, the executed tests are distributed on more than a hundred physical hosts. Furthermore, since SAP HANA can also be used in a distributed setup, some tests require multiple hosts on their own. As distributed testing systems are commonly used for large software projects in the software industry [1, 29], we analyze potential environmental characteristics that could lead to test flakiness, with the goal of better understanding the impact of the test environment on flaky failures.



(a) Testing stages of SAP HANA.

(b) Data collection process for MT dataset [8].

Figure 1: Study subject and data collection.

Table 1: Available datasets for this study.

Dataset	# Tests	# Test Executions
PE	721	1 528 986
MT	688	129 660

3.2 Datasets

For SAP HANA, the results of all test executions are stored in a database, together with metadata about the tests and the execution hosts. In this study, we utilize this data to construct two different datasets.

First, in the *Production Environment* dataset (PE), we collect production data from SAP HANA's CI pipeline. That is, we collect the results of system tests executed in SAP HANA's pre-submit testing stage within a 12-week period. As a result, we end up with more than 1.5 million test executions.

Second, we collect data from our so-called *Mass Testing* (MT) experiment [8]. The process of the experiment is illustrated in Figure 1b. The idea of the mass testing experiment is to repeatedly execute SAP HANA's pre-submit tests on the same version of the code to study test flakiness. To create the dataset, we used idle resources in the testing infrastructure on weekends. Our objective is to aggregate 100 repeated test executions per code version. To achieve this, we run a job that executes SAP HANA's pre-submit tests 20 times for the same code version every weekend. The job first checks if there already exist 100 test repetitions with the current version of the code. If so, the code is updated to the most recent version of the main code line. If there are less than 100 repetitions, the job executes the tests to collect data from 20 additional repetitions. Based on this process, we have collected more than 100 000 test executions in the same 12-week period as for the PE dataset. We provide an overview of the datasets in Table 1.

3.3 Flakiness Definition

The traditional informal definition of flakiness classifies a test as flaky when it yields different results for repeated executions on the same version of the code [31]. Previous research examined different features associated with this binary notion of flakiness to predict whether a test is flaky [7, 9, 18, 23, 32, 33]. However, previous research also pointed out that, especially in large testing environments, quantifying flakiness beyond this binary notion can be useful, as all tests exhibit some level of flakiness [21]. In a previous study at SAP HANA, we found that most system tests in SAP HANA's pre-submit test suite show some degree of flakiness when the number of test repetitions grows towards infinity [8]. Therefore, for this work, rather than relying on the binary notion of flakiness, we examine the failure rate of flaky tests [8, 36].

More formally, given a test t we compute $\mathcal{R}_p(t)$, the *flakiness rate* of t in period p, by dividing the number of flaky executions $f_p(t)$ by the total number of executions $e_p(t)$ of that test t in period p. Here, we label an execution as flaky if the test failed on a certain version of the code for which it also showed passing results. Thus, the formula for calculating the flakiness rate $\mathcal{R}_p \in [0, 1]$ of t in p is:

$$\mathcal{R}_p(t) = \frac{f_p(t)}{e_p(t)}.$$

This approach allows us to quantify the effect that different characteristics have on the frequency of flaky failures beyond a binary flakiness definition. It further allows us to estimate the additional computational cost caused by flaky tests because the costs of re-executions increase with the frequency of flaky failures [8, 21].

3.4 Research Questions and Methods

Our study is based on two main research questions. In the following, we introduce and motivate these questions and outline the methods we use to answer them.

> **RQ1**: How do different test complexity characteristics correlate with test flakiness in the context of SAP HANA?

Previous research indicates that higher test complexity can lead to a higher flakiness rate [1, 2, 8, 21]. To answer **RQ1**, we evaluate various test characteristics related to test complexity to check

Table 2: Test and environment characteristics: We hypothesize that each of these characteristics has an impact on the flakiness rate of tests in the context of SAP HANA.

Category	Characteristic	Related work
Test	Test execution time	[2, 26]
	CPU	[13, 27, 36]
	Memory	[36]
	Distributed test	None
Test environment	Host performance	None
	System load	[13, 27, 36]

whether these characteristics have a significant effect on the flakiness rate of tests in the context of SAP HANA. Some of our hypotheses are derived from related work, while others are the result of internal discussions with practitioners at SAP. Table 2 summarizes the characteristics that we examined and, if available, the previous work that motivates them. In the following, we discuss the motivation for the selected characteristics in detail.

On test execution time: Previous research has found that test execution time can be an effective feature to predict whether a test is flaky [2]. However, the reasons for flakiness have been shown to vary between different types of software [19] and between programming languages [6]. Therefore, in this study, we examine the relationship between test execution time and flakiness rate in the context of SAP HANA, a large-scale database management system written mainly in C++.

We first validate the findings of previous research suggesting that test execution time is correlated with test flakiness [2]. We further quantify the strength of this relationship with the help of *Pearson's correlation coefficient r* and report the corresponding p-value with a significance threshold of $p \leq 0.05$.

On required resources: An article in Google's testing blog states that the memory usage of a test explains part of the variance in its flakiness rate (r2=0.76) [28]. The article further reports that tests labeled "large" by developers show a higher flakiness rate than tests labeled to be of "medium" size, which, in turn, are more likely to be flaky than tests labeled as "small" [28]. In the SAP HANA organization, developers must provide detailed metadata on the resources required for running a test. As described in Section 3.1, for each group of tests, developers assign a certain host configuration that contains the number of available CPU threads and memory.

Based on the findings reported in the Google article and previous research that found a relationship between flakiness and CPU/memory constraints [13, 27, 36], we hypothesize that tests requiring more computational resources in terms of CPU or memory show a higher flakiness rate. To validate this hypothesis, we group the tests based on their required resources as labeled by SAP HANA's developers and check whether higher memory or CPU requirements correlate with the tests' flakiness rates.

On distributed environments: As mentioned in Section 3.1, some tests in SAP HANA's pre-submit stage require a distributed setup because they verify distributed functionality (e.g., distributed transactions [17]). As motivated above, tests in distributed environments might be more susceptible to common flakiness categories. Therefore, we hypothesize that such *distributed tests* show a higher flakiness rate than non-distributed tests.

> **RQ2**: How do different test environment characteristics correlate with test flakiness in the context of SAP HANA?

Previous research claims that more complex testing environments can lead to more flakiness [1, 8, 21]. In this research question, our aim is to verify this claim by breaking the complexity of SAP HANA's testing environment down into concrete characteristics of the environment as listed in Table 2. To answer **RQ2**, we use a similar method as for **RQ1**: we check whether environmental characteristics show a significant correlation with the flakiness rate of SAP HANA's system tests. In the following, we motivate our selection of environmental characteristics.

On execution host performance: As mentioned in Section 3.1, SAP HANA's testing infrastructure consists of approximately 1000 heterogeneous hosts, i.e., physical servers located in SAP's data centers. These hosts were purchased in waves within the last ten years and vary in terms of their computational performance, for example, due to differences in the built-in processor generation. Based on our assumption that the flakiness rate of a test increases with test execution time, as described for **RQ1**, we hypothesize that the flakiness rate on a host decreases with the host's performance.

On system load: Previous research has shown that tests might fail flakily due to resource limits [27, 36]. We further investigate the idea of resource limits impacting flakiness by assessing the relationship between system load and test flakiness. To achieve this, we operationalize the load on our testing infrastructure as the number of tests executed per hour. We then check whether this notion of system load correlates with the flakiness rate of the executed tests.

4 Empirical Results

In this section, we present our results along the research questions and the characteristics motivated above.

4.1 RQ1: Test Complexity

To answer **RQ1**, we evaluate different test complexity characteristics and their correlation with the flakiness rate of SAP HANA's system tests. In the following, we present our results for each of the characteristics listed in Section 3.4.

4.1.1 Average test execution time. Based on the assumption that longer-running tests yield higher flakiness rates, we calculate the average execution time for each of SAP HANA's system tests. To this end, we first exclude outliers by removing test executions with execution times that are outside the $10th$ to $90th$ percentile in the period considered. We then calculate the arithmetic mean of the remaining execution times and examine their relationship with the flakiness rate of the respective test as defined in Section 3.3.

To ensure a reasonable sample size of test executions per test for our analysis, we remove tests from the PE dataset that were executed less than 50 times in the given period. In the filtered dataset, the average number of executions per test is 1752.

To further reduce the impact of outliers in our dataset, we divide the tests into equal-width bins based on their execution time, i.e.,

Table 3: Pearson's correlation coefficient (r) of average test execution time and flakiness rate together with p-values.

Dataset	r	p-value
PE	0.74	0.0063
MT	0.79	0.0022

Table 4: Equal-width execution time bins and the respective number of tests per bin for the two datasets.

Interval	# Tests PE	# Tests MT
(0, 5]	376	361
(5, 10]	90	88
(10, 15]	72	65
(15, 20]	55	57
(20, 25]	32	23
(25, 30]	18	24
(30, 35]	22	22
(35, 40]	16	14
(40, 45]	11	17
(45, 50]	7	7
(50, 55]	3	4
(55, 60]	4	4

every bin contains tests for which the average execution time lies within a 5-minute interval. Since the execution times range from a few seconds to almost one hour, we end up with 12 bins. Table 4 shows the resulting bins and the number of tests per bin.

Figure 2 visualizes the relationship between average test execution time and flakiness rate. The orange line represents the regression line, which we obtain by fitting a *linear regression model*. To quantify the strength of the relationship, we calculate *Pearson's correlation coefficient* r together with the corresponding p-value using the `pearsonr` function from the SciPy library with default settings [12]. We report the r values for the two datasets separately using indices (e.g., r_{PE} is the correlation coefficient in the PE dataset).

As shown in Table 3, the correlation coefficients for the two datasets are $r_{PE} = 0.74$ and $r_{MT} = 0.79$, suggesting that there exists a positive correlation between average test execution time and flakiness rate. As the resulting p-values as shown in Table 3 are below our threshold of 0.05, we conclude that the flakiness rate is indeed correlated with the average test execution time.

Answer RQ1 (execution time): There exists a significant positive correlation between average test execution time and flakiness rate in both datasets. The correlation coefficient of this relationship is 0.74 for the PE and 0.79 for the MT dataset.

4.1.2 Required resources. In the following, we investigate the relationship between a test's flakiness rate and its resource requirements as labeled by SAP HANA's developers. As mentioned in Section 3.1, developers assign their tests to a certain pool of hosts

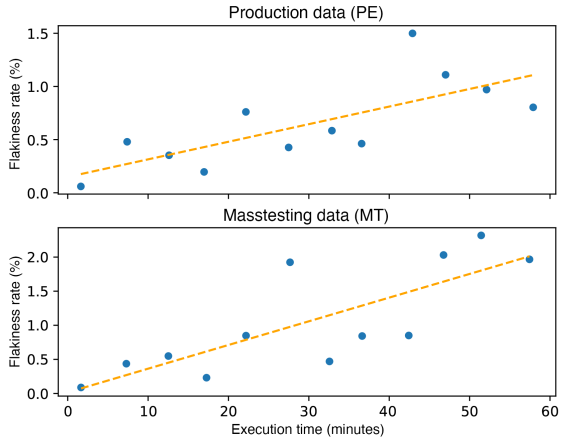


Figure 2: Scatterplot showing the relation between test execution time and mean flakiness rate. The orange line depicts the fitted regression line.

that satisfy the required computational resources. Every pool is defined by the available CPU threads and main memory per host. We use these labels to investigate whether "larger" tests show higher flakiness rates.

For our analysis, we group the tests in the two datasets according to the assigned memory and CPU requirements, respectively. Based on the resulting groups, we calculate the flakiness rate of each group and test whether there exists a positive correlation between the flakiness rate and the respective memory or CPU labels.

Figure 3 shows the average test flakiness rates in relation to the assigned memory labels for the PE and MT datasets. We find that there exists a significant positive correlation between assigned memory label and flakiness rate in the PE dataset ($r_{PE} = 0.47$, $p = 0.05$). In the MT dataset, however, we observe no significant correlation between flakiness rate and assigned memory label ($r_{MT} = -0.14$, $p = 0.6$).

As visible in Figure 4, these findings also apply to the assigned CPU labels. Although there exists a significant positive correlation between assigned CPU labels and flakiness rates in the PE dataset ($r_{PE} = 0.69$, $p = 0.03$), we do not observe a significant relationship in the MT dataset ($r_{MT} = -0.13$, $p = 0.6$).

Answer RQ1 (resources): The relationship between the flakiness rate and the required test resources is inconclusive. Although there exist significant correlations between the flakiness rate and both the number of assigned CPU threads and the assigned main memory in the PE dataset, we do not observe any significant correlations in the MT dataset.

4.1.3 Distributed tests. In the following, we investigate whether distributed tests are more flaky than tests that do not require distributed environments. As explained in Section 3.1, all test executions and their results with additional metadata are stored in a database. The metadata about test executions contains a label on

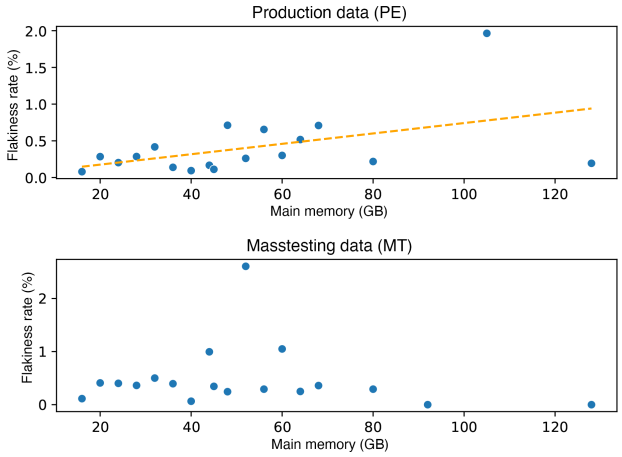


Figure 3: Scatterplot showing the relation between available memory and mean flakiness rate. Significant positive correlation in PE dataset, no significant correlation in MT dataset.

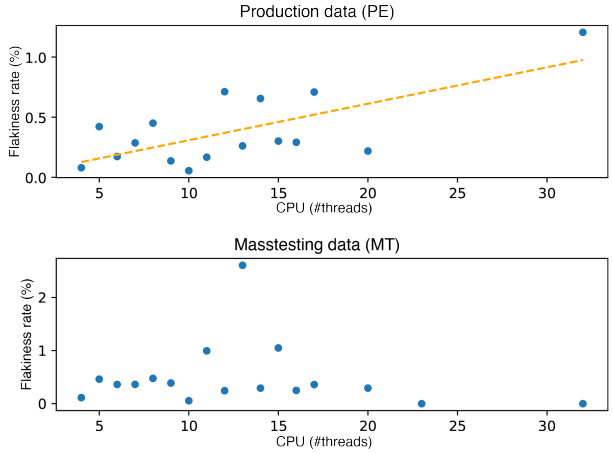


Figure 4: Scatterplot showing relation between available CPU threads and mean flakiness rate. Significant positive correlation in PE dataset, no significant correlation in MT dataset.

whether the test requires a distributed environment in its setup, which allows us to examine whether such distributed tests have an increased flakiness rate.

For our analysis, we divide the tests into two groups, based on the binary label "is distributed". Overall, approximately 6 % of SAP HANA's pre-submit tests in our study belong to the distributed group. We then calculate the average flakiness rate for both groups and compare them. Finally, we conduct a *Mann-Whitney U test* as implemented by SciPy. The Mann-Whitney U test is a nonparametric test of the null hypothesis that the underlying distribution of two samples is the same [11]. In our case, we use Mann-Whitney U to determine whether the distribution of flakiness rates for distributed tests is significantly different from the flakiness rates for non-distributed tests [11].

As shown in Figure 5, the results differ between the two datasets. While distributed tests appear more flaky in the MT datasets, the opposite is true for the PE dataset. Looking at individual tests and their flakiness rates, we found that the distributed test with the highest flakiness rate is the same for both datasets. The test verifies distributed streaming functionality and faced an *async wait* issue leading to flaky failures. However, this issue was fixed by a developer within the 12-week period we consider for this study. As we updated the code version in our mass testing experiment only after 100 test repetitions, i.e., every fifth week, the issue caused a high number of flaky failures and led to a flakiness rate of 17.1 % in the MT dataset, which heavily influenced the mean of the distributed tests. Removing this test from the data decreases the mean flakiness rate for the distributed tests in MT from 0.8 to 0.4, which then leads to a result similar to that in the PE dataset.

Looking at the results of our *Mann-Whitney U test*, we find that the flakiness rate of distributed tests is significantly lower for distributed tests in the PE dataset ($p = 5.23 \times 10^{-13}$). However, the difference is not significant in the MT dataset ($p = 0.97$).

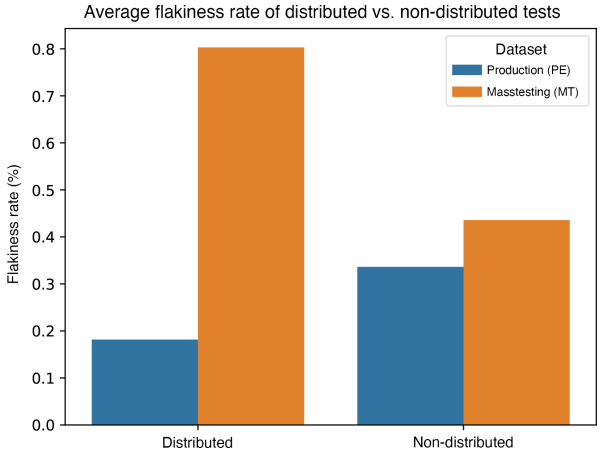


Figure 5: Arithmetic mean of test flakiness rate in both datasets, tests divided by the "is distributed"-label. Note that the mean for distributed tests on the MT dataset was heavily influenced by a single test with a flakiness rate of 17.1%.

> **Answer RQ1 (distributed tests):** In the given datasets, we do not observe an increase in the flakiness rate when testing distributed environments. In contrast, we observe that distributed tests show a significantly lower flakiness rate than non-distributed tests in the PE dataset.

4.2 RQ2: Test Environment

To answer **RQ2**, we assess the correlation of several characteristics in SAP HANA's testing environment (see Section 3.4) with the flakiness rate of system tests only based on production data (PE), because our experimental data (MT) does not contain enough information about the environment.

4.2.1 Execution Host Performance. First, we investigate the correlation between the execution host's computational performance and the flakiness rate.

As mentioned in Section 3.1, SAP HANA's testing infrastructure consists of approximately 1000 hosts. These hosts are grouped into pools, depending on their available resources. In the scheduling phase, the tests are executed in Docker containers, which run on the respective hosts, depending on their pool label. These pool labels define CPU and main memory requirements, which are then granted to the respective Docker container. Due to this virtual layer handling resource assignments, each physical host can belong to n pools, as long as it fulfills the required resources for the pool configuration. Furthermore, each pool is assigned m hosts.

Once a test run is started, the tests are scheduled on the different hosts by a load balancer to make sure that the load is evenly distributed over the infrastructure. As the hosts have different resource configurations, the set of tests that the hosts execute differs. Therefore, we cannot simply compare the average test execution times per host to assess whether they influence the flakiness rates.

To mitigate this problem, we scale the execution times *within-test* using sklearn's *MinMaxScaler* [35]. That is, let $E = \{e \in \mathbb{R}_0^+\}$ be the set of execution times of a certain test. After filtering out outliers by focusing on values between the 10th and 90th percentile, we scale each execution time $e \in E$ using the following formula: $\hat{e} = \frac{e - min(E)}{max(E) - min(E)} \in [0, 1]$. Thus, $\hat{E} = \hat{e} \in [0, 1]$ contains values in $[0, 1]$ for each test. We use the average of these scaled execution times as performance indicators for our hosts. Intuitively, the closer $\hat{e}$ for a given execution is to 1, the closer the execution time is to the maximum execution time of the respective test in E. Figure 7 shows the resulting distribution of host performance indicators, where a higher performance indicator implies that tests yield longer execution times on the respective host.

To examine the relationship between host performance and flakiness rate, we group test executions based on the execution host and calculate average flakiness rates per host. According to our flakiness definition in Section 3.3, we label an execution as flaky if it led to a failure and there exists an execution of the same test on the same version of the code that passed, although this passing run might have been executed on a different host. Similar to our approach in Section 4.1, we group hosts into 100 equal-width bins based on their performance indicator.

Figure 6 shows the relationship between the host performance indicator and the flakiness rate. There exists a significant positive correlation ($r_{PE} = 0.74$). Since the p-value is below our threshold of 0.05 ($p = 4.45 \times 10^{-13}$), we conclude that the flakiness rate increases significantly when the performance indicator of an execution host increases, i.e., when tests run longer on average on that host.

Answer RQ2 (host performance): There exists a significant positive correlation between the host performance indicator we defined (mean filtered and scaled test execution time per host) and the flakiness rate. The correlation coefficient for this relationship is 0.74 ($p < 0.05$), which indicates that the flakiness rate increases when the computational performance of the execution host decreases.

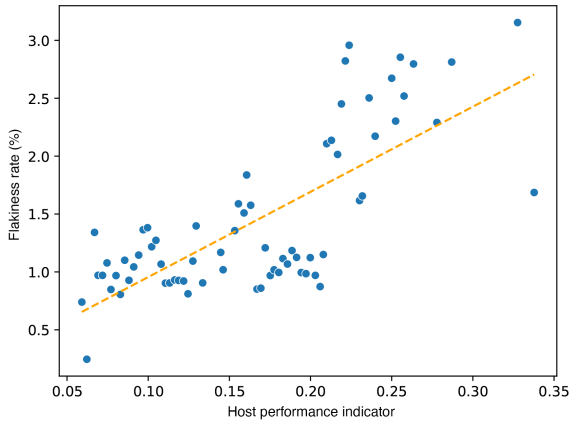


Figure 6: Scatterplot showing the relation between host performance indicator and mean flakiness rate ($r_{PE} = 0.74$). The orange line depicts the result of fitting a linear regression.

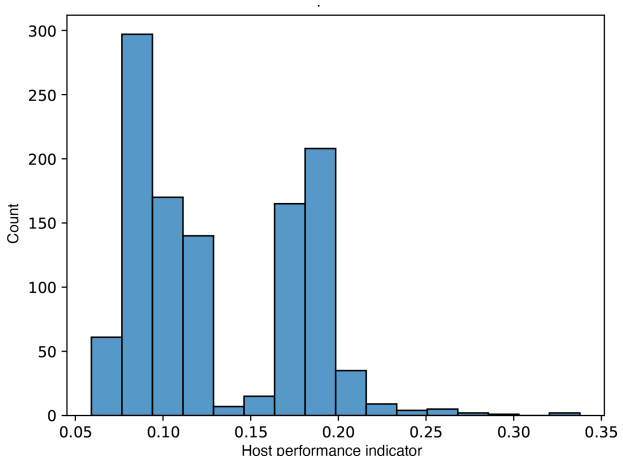


Figure 7: Histogram showing host performance indicator distribution (average scaled test execution time per host).

4.3 System Load

In the following, we report the results of our correlation analysis on system load and flakiness rate. As mentioned in section 3.4, we operationalize the load on our testing system as the number of tests that were executed per hour.

Figure 8b illustrates the idea of this operationalization with the example of a single day in the given dataset. As Figure 8b shows, the peak of the system load on this day occurs at 18:00 (6pm) Central European Time (CET), which is intuitive, as developers typically merge their latest changes before the end of their work day. The number of test executions per hour on that day ranges from 1363 at 4:00 (4am) to 24 443 at 18:00 (6pm) with a mean value of 6381. As SAP's headquarters are located in Germany, most developers work in the CET timezone. However, part of the development is also done in different time zones, which explains the local maximum in the early morning hours.

To analyze the relationship between system load and flakiness rate, we group the given samples into 100 equal-width bins based on their load, similar to our approach in Section 4.1. We then calculate the average flakiness rate per bin. The resulting average flakiness rates are shown in Figure 8a. As Figure 8a shows, there is no (linear) relationship between the system load and the flakiness rate. In fact, *Pearson's correlation coefficient* is $r_{PE} = 0.08$ with a p-value of 0.91. Therefore, we conclude that there is no linear relationship between system load and flakiness rate.

Answer RQ2 (system load): In the data we analyzed, there is no significant correlation between the system load (executed tests per hour) and the flakiness rate.

5 Discussion

We discuss the practical implications of our empirical results.

On test execution time: Our analysis in the context of SAP HANA revealed a significant positive correlation between the average execution time of a test and its flakiness rate. This finding is in line with previous research that identified test execution time as an effective feature to predict whether a test is flaky [2, 26]. As previous research has argued, all tests can be considered flaky [8, 20] because every test might fail flakily due to interruptions of its execution context by some external event. Assuming that such interrupting events are uniformly distributed over time, this would explain a linear relationship between the execution time of a test and its flakiness rate. For example, for tests requiring a network connection, the rate of flaky failures due to network outages grows with the time that they depend on the network during execution. As the test execution time increases, the probability of a network outage occurring at some point during that execution time also increases.

To mitigate the problem caused by this relationship, one common strategy is to split long-running tests to decrease their flakiness [4]. While splitting a single long-running test tends to decrease the flakiness rate, it also increases the effectiveness of re-executing flaky tests, as the scope of re-executions is narrowed down to the actual point of failure. However, executing multiple tests after splitting one larger test might cause additional overhead due to test setup times. Therefore, developers must find an appropriate trade-off between a narrow scope for efficient re-executions of flaky failures and a reduction of the required setup times for the separated tests.

On required computational resources: Unexpectedly, the relationship between the computational resources required for a test and its flakiness rate varies between the datasets we used in our study. Although a significant positive correlation exists in the PE dataset between the flakiness rate and the required CPU threads or the required memory, respectively, we did not observe significant correlations in the MT dataset. We assume that the correlation in the PE dataset is a spurious correlation, which was confounded by a different aspect that we did not consider in this study. Future work is required to investigate potential confounding factors.

On testing distributed environments: Contrary to our expectations, we found that distributed tests actually showed a lower flakiness rate than non-distributed tests. One possible explanation could be that developers implementing distributed tests have a special focus on the consistency of their tests due to the inherent consistency discussion around distributed systems. In fact, the distributed test that yielded the highest flakiness rate in our experiment was fixed during our study period.

Fixing flaky tests is often a tedious task for developers, as flaky failures can be difficult to reproduce [24, 25]. As noted in previous research, fixing flaky tests usually has a lower priority compared to tests with permanent failures [15]. However, due to the consistency requirements of functionality in distributed systems, we expect that resolving or reducing flaky failures in distributed tests will gain increasing attention.

On execution host performance: In the data we analyzed, we observe a significant positive correlation between our host performance indicator and the flakiness rate. Based on our host performance definition, we conclude that flakiness rates increase when tests are executed on hosts with a lower performance. This finding is in line with our finding for **RQ1**. The longer tests are executed, the higher their flakiness rate, regardless of the reasons for the long execution time. Long execution times can be caused by the scope of the test itself or a slow execution environment.

To mitigate the problems caused by the high variance in execution times on heterogeneous hosts in SAP HANA's testing environment, our previous research on reducing timeout flakiness [8] has motivated the introduction of a global static timeout value of 2 hours for each test execution. When collecting the data for this study, this global static timeout value was already active. Therefore, we assume that most flaky failures in our study were not caused by flaky timeouts but by other forms of flakiness.

On system load: In our analysis, we did not find a significant relationship between the flakiness rate and the load on the testing system. We shared this finding with practitioners at SAP and received the feedback that this might indicate that the load-balancing of SAP HANA successfully distributes tests to prevent exceeding resource limits.

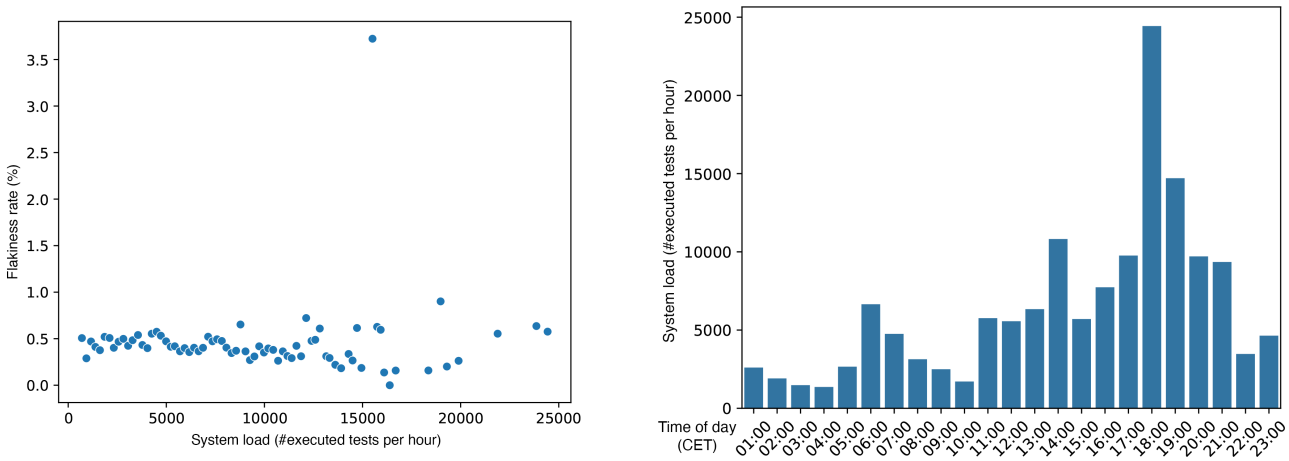
6 Threats to Validity

We discuss threats to the construct, internal, and external validity.

6.1 Construct Validity

We discuss the degree to which our operationalizations of the investigated characteristics measure the intended properties [34].

On required computational resource labels: As mentioned in Section 3.4, SAP HANA developers need to provide a label that specifies the resources required to run a test in terms of available CPU threads and main memory. However, from an organizational perspective, SAP HANA's testing budget limits the runtime (i.e., clock time) that developers can spend running their tests rather than limiting the amount of computational resources the tests consume [4]. As a result, labeling tests with higher resource requirements than needed does not affect the testing budget that developers have. On the contrary: running tests on pools with higher performance might even reduce the test runtime and hence the impact on the testing budget. This, in turn, might lead to "over-sized" resource labels, which do not reflect the actual required amount of computational resources that the tests need.



(a) Scatterplot: Mean flakiness rate vs. system load.

(b) Barplot: Distribution of the system load for a single day.

Figure 8: Relationship of system load and mean flakiness rate together with system load over an exemplary day.

On the operationalization of system load: In this study, we approximate the load on SAP HANA's testing infrastructure by counting the number of tests that were executed in a given period, i.e. one hour. This measure does not account for the scope and size of the executed tests, which might bias the results. However, since the executed test suites are relatively stable over time, we expect the scope and size of the tests to be more or less equally distributed across the intervals. Beyond the context of our study, the number of tests executed in a certain period is commonly used as a proxy to monitor the system load at SAP HANA. However, the chosen interval of one hour could lead to a loss of information since peaks that occur for a short time might not be visible.

6.2 Internal Validity

We discuss the degree to which we can dismiss alternative explanations for our results [10].

As we draw our conclusions based on an isolated analysis of the flakiness rate and certain characteristics of our tests, our analysis might be vulnerable to confounding variables that influence the examined relationships. To mitigate this threat, we repeated our analyses on two different datasets, one based on production data (PE) and one based on targeted experiments (MT). However, for future work, we encourage studies to investigate more sophisticated relationships and interactions between different test and test environment characteristics.

6.3 External Validity

We discuss how our results generalize to other projects [5].

Due to the specific context of SAP HANA, our results are restricted to this context and may not generalize to other projects. Previous research has also shown that the contributing factors for flakiness vary between different types of projects. Therefore, we encourage further studies targeting other industrial software systems to validate our findings.

7 Conclusion

We conducted a study of test flakiness in the context of a large industrial database management system, SAP HANA. Motivated by previous research and internal discussions at SAP, we tested a range of test and environmental characteristics regarding their relationship with test flakiness. Instead of relying on the traditional binary notion of flakiness, we investigate whether these characteristics increase the flakiness rate of tests.

Our study reveals that there exists a significant positive correlation between the flakiness rate and the average execution time of a test ($r_{PE} = 0.74$, $r_{MT} = 0.79$). According to our findings, this is true not only when the scope of a test causes longer execution times but also when it is caused by a slow infrastructure. Moreover, we found that tests that require distributed setups are significantly less flaky than tests that were executed on a single host. We attribute this finding to the increased priority that distributed systems developers assign to problems related to consistency.

In contrast to a previous industry report by Google, we found the relationship between the flakiness rate of a test and its required resources to be inconclusive. We expect that there exist confounding factors that influence this relationship, and thus encourage future studies to further examine the relationship between flaky tests and the computational resources a test requires. Interestingly, we did not find a significant correlation between the load on SAP HANA's testing infrastructure and test flakiness. When we shared this finding with practitioners at SAP HANA, they interpreted it as a confirmation that the load balancer successfully distributes tests across the infrastructure. Furthermore, this finding leads to the conclusion that the current scale of the testing infrastructure is capable of handling the immense computational demands of SAP HANA's continuous integration pipelines.

Since the results of our study are in the unique context of SAP HANA, we encourage future studies to investigate how our findings generalize to other industrial or open-source software systems.

References

[1] Nadia Alshahwan, Mark Harman, and Alexandru Marginean. 2023. Software Testing Research Challenges: An Industrial Perspective. In *2023 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 1–10. https://doi.org/10.1109/ICST57152.2023.00008

[2] Abdulrahman Alshammari, Christopher Morris, Michael Hilton, and Jonathan Bell. 2021. Flakeflagger: Predicting Flakiness without Rerunning Tests. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 1572–1584.

[3] Thomas Bach, Artur Andrzejak, Changyun Seo, Christian Bierstedt, Christian Lemke, Daniel Ritter, Dong Won Hwang, Erda Sheshi, Felix Schabernack, Frank Renkes, et al. 2022. Testing Very Large Database Management Systems: The Case of SAP HANA. *Datenbank-Spektrum* 22, 3 (2022), 195–215.

[4] Thomas Bach, Ralf Pannemans, and Sascha Schwedes. 2018. Effects of an Economic Approach for Test Case Selection and Reduction for a Large Industrial Project. In *2018 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE, IEEE Computer Society, 374–379.

[5] Sebastian Baltes and Paul Ralph. 2022. Sampling in software engineering research: A critical review and guidelines. *Empirical Software Engineering* 27, 4 (2022), 94.

[6] Keila Barbosa, Ronivaldo Ferreira, Gustavo Pinto, Marcelo d'Amorim, and Breno Miranda. 2022. Test Flakiness Across Programming Languages. *IEEE Transactions on Software Engineering* 49, 4 (2022), 2039–2052. https://doi.org/10.1109/TSE.2022.3208864

[7] Jonathan Bell, Owolabi Legunsen, Michael Hilton, Lamyaa Eloussi, Tifany Yung, and Darko Marinov. 2018. DeFlaker: Automatically Detecting Flaky Tests. In *Proceedings of the 40th International Conference on Software Engineering (ICSE)*. ACM, New York, NY, USA, 433–444.

[8] Alexander Berndt, Sebastian Baltes, and Thomas Bach. 2024. Taming Timeout Flakiness: An Empirical Study of SAP HANA. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice, ICSE-SEIP 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, New York, NY, USA, 69–80. https://doi.org/10.1145/3639477.3639741

[9] Alexander Berndt, Zoltán Nochta, and Thomas Bach. 2023. The Vocabulary of Flaky Tests in the Context of SAP HANA. In *2023 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. IEEE, 1–9. https://doi.org/10.1109/ESEM56168.2023.10304860

[10] Marilynn B Brewer and William D Crano. 2000. Research Design and Issues of Validity. *Handbook of research methods in social and personality psychology* (2000), 3–16.

[11] SciPy Community. 2024. *SciPy Docs - Mann-Whitney U*. SciPy. https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.mannwhitneyu.html Accessed 2024-07-31.

[12] SciPy Community. 2024. *SciPy Docs - Pearsonr*. SciPy. https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.pearsonr.html Accessed 2024-07-26.

[13] Marcello Cordeiro, Denini Silva, Leopoldo Teixeira, Breno Miranda, and Marcelo d'Amorim. 2021. Shaker: a tool for detecting more flaky tests faster. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, IEEE, 1281–1285.

[14] Thomas Durieux, Claire Le Goues, Michael Hilton, and Rui Abreu. 2020. Empirical Study of Restarted and Flaky Builds on Travis CI. In *Proceedings of the 17th International Conference on Mining Software Repositories (MSR)*. Association for Computing Machinery, New York, NY, USA, 254–264.

[15] Moritz Eck, Fabio Palomba, Marco Castelluccio, and Alberto Bacchelli. 2019. Understanding Flaky Tests: The Developer's Perspective. In *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/SIGSOFT FSE 2019, Tallinn, Estonia, August 26-30, 2019*, Marlon Dumas, Dietmar Pfahl, Sven Apel, and Alessandra Russo (Eds.). ACM, 830–840. https://doi.org/10.1145/3338906.3338945

[16] Emad Fallahzadeh and Peter C Rigby. 2022. The Impact of Flaky Tests on Historical Test Prioritization on Chrome. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 273–282.

[17] Franz Färber, Norman May, Wolfgang Lehner, Philipp Große, Ingo Müller, Hannes Rauhe, and Jonathan Dees. 2012. The SAP HANA Database–An Architecture Overview. *IEEE Data Eng. Bull.* 35, 1 (2012), 28–33.

[18] Sakina Fatima, Taher A Ghaleb, and Lionel Briand. 2022. Flakify: A Black-box, Language Model-based Predictor for Flaky Tests. *IEEE Transactions on Software Engineering* 49, 4 (2022), 1912–1927.

[19] Martin Gruber and Gordon Fraser. 2022. A Survey on How Test Flakiness Affects Developers and What Support They Need to Address it. In *2022 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, IEEE, 82–92.

[20] Mark Harman and Peter O'Hearn. 2018. From Start-ups to Scale-ups: Opportunities and Open Problems for Static and Dynamic Program Analysis. In *2018 IEEE 18Th international working conference on source code analysis and manipulation (SCAM)*. IEEE, IEEE, 1–23.

[21] Emily Kowalczyk, Karan Nair, Zebao Gao, Leo Silberstein, Teng Long, and Atif M. Memon. 2020. Modeling and Ranking Flaky Tests at Apple. In *ICSE-SEIP 2020: 42nd International Conference on Software Engineering, Software Engineering in Practice, Seoul, South Korea, 27 June - 19 July, 2020*, Gregg Rothermel and Doo-Hwan Bae (Eds.). ACM, 110–119. https://doi.org/10.1145/3377813.3381370

[22] Wing Lam, Kivanç Muslu, Hitesh Sajnani, and Suresh Thummalapenta. 2020. A Study on the Lifecycle of Flaky Tests. In *ICSE '20: 42nd International Conference on Software Engineering, Seoul, South Korea, 27 June - 19 July, 2020*, Gregg Rothermel and Doo-Hwan Bae (Eds.). ACM, 1471–1482. https://doi.org/10.1145/3377811.3381749

[23] Wing Lam, Reed Oei, August Shi, Darko Marinov, and Tao Xie. 2019. iDFlakies: A Framework for Detecting and Partially Classifying Flaky Tests. In *2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST)*. IEEE, IEEE, 312–322.

[24] Wing Lam, Stefan Winter, Angello Astorga, Victoria Stodden, and Darko Marinov. 2020. Understanding Reproducibility and Characteristics of Flaky Tests through Test Reruns in Java Projects. In *2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, IEEE, 403–413. https://doi.org/10.1109/ISSRE5003.2020.00045

[25] Wing Lam, Stefan Winter, Anjiang Wei, Tao Xie, Darko Marinov, and Jonathan Bell. 2020. A large-scale longitudinal study of flaky tests. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 1–29.

[26] Johannes Lampel, Sascha Just, Sven Apel, and Andreas Zeller. 2021. When Life Gives you Oranges: Detecting and Diagnosing Intermittent Job Failures at Mozilla. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. Association for Computing Machinery, New York, NY, USA, 1381–1392.

[27] Fabian Leinen, Alexander Perathoner, and Alexander Pretschner. 2024. On the Impact of Hitting System Resource Limits on Test Flakiness. In *2024 International Flaky Tests Workshop 2024 (FTW'24)*. Association for Computing Machinery, New York, NY, USA, 14–19.

[28] Jeff Listfield. 2020. *Where do our Flaky Tests Come From?* Google. https://testing.googleblog.com/2017/04/where-do-our-flaky-tests-come-from.html 2024-07-14, archived by Internet Archive at http://web.archive.org/web/20221113232600/https://testing.googleblog.com/2017/04/where-do-our-flaky-tests-come-from.html.

[29] Atif M. Memon, Zebao Gao, Bao N. Nguyen, Sanjeev Dhanda, Eric Nickell, Rob Siemborski, and John Micco. 2017. Taming Google-Scale Continuous Testing. In *39th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice Track, ICSE-SEIP 2017, Buenos Aires, Argentina, May 20-28, 2017*. IEEE Computer Society, 233–242. https://doi.org/10.1109/ICSE-SEIP.2017.16

[30] John Micco. 2017. *The State of Continuous Integration Testing at Google*. https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/45880.pdf ICST Keynote, accessed 2024-09-07.

[31] Owain Parry, Gregory M Kapfhammer, Michael Hilton, and Phil McMinn. 2021. A Survey of Flaky Tests. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 31, 1 (2021), 1–74. https://doi.org/10.1145/3476105

[32] Gustavo Pinto, Breno Miranda, Supun Dissanayake, Marcelo d'Amorim, Christoph Treude, and Antonia Bertolino. 2020. What is the Vocabulary of Flaky Tests?. In *Proceedings of the 17th International Conference on Mining Software Repositories (MSR)*. Association for Computing Machinery, New York, NY, USA, 492–502.

[33] Valeria Pontillo, Fabio Palomba, and Filomena Ferrucci. 2022. Static Test Flakiness Prediction: How far can we go? *Empirical Software Engineering* 27, 7 (2022), 187.

[34] Paul Ralph and Ewan Tempero. 2018. Construct Validity in Software Engineering Research and Software Metrics. In *Proceedings of the 22nd International Conference on Evaluation and Assessment in Software Engineering (EASE 2018)*. Association for Computing Machinery, New York, NY, USA, 13–23.

[35] scikit Learn. 2024. *scikit-learn Docs*. Scikit-Learn. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html Accessed on 2024-07-28.

[36] Denini Silva, Martin Gruber, Satyajit Gokhale, Ellen Arteca, Alexi Turcotte, Marcelo d'Amorim, Wing Lam, Stefan Winter, and Jonathan Bell. 2023. The Effects of Computational Resources on Flaky Tests. *arXiv preprint arXiv:2310.12132* abs/2310.12132 (2023).